Real-Time Multi-Core Sorting and Median Filter Service

A Report

Submitted for the course

26ELC331 Real Time Operating System using QNX

by

Anmol Govindarajapuram Krishnan (CB.EN.U4EEE23103)

Mauli Rajguru (CB.EN.U4EEE23120)

Sanjay TS (CB.EN.U4EEE23136)



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 17, 2026

Contents

List of Figures iii

List of Tables v

Abbreviations vi

List of Symbols vii

1 Introduction 1

1.1 Background and Motivation 1

1.2 Project Objectives 1

1.3 Workload and Dataset 2

1.4 Report Structure 3

2 System Overview 4

2.1 Introduction 4

2.2 System Architecture and Block Diagram 4

2.3 Client-Server Communication Sequence 6

2.4 Data Flow Diagram 9

3 Methodology 11

3.1 Introduction 11

3.2 Virtual Sensor 11

3.2.1 Configuration 11

3.2.2 Addressing a sample in either layout 12

3.2.3 The read-ahead window 12

3.2.4 Frames, blocks and the sensor's clock 13

3.2.5 Rejecting bad input, and the interface 14

3.3 Block Loader . . . 14
3.4 Parallel Merge-Sort . . . 15
3.4.1 The pool and the plan . . . 15
3.4.2 The merge, and why the answer is stable . . . 15
3.4.3 The barrier between phases . . . 17
3.4.4 Refusals, and the plan line . . . 17
3.4.5 Measured scaling . . . 18
3.5 Rolling Median Filter (Hampel Filter) . . . 18
3.6 QNX IPC Mechanisms . . . 19
3.6.1 Message Passing and IOV . . . 19
3.6.2 QNX Pulses and Asynchronous Notification . . . 19
3.6.3 Shared Memory . . . 19
3.7 Testing . . . 19
3.7.1 Two folds, one set of sources . . . 19
3.7.2 Four kinds of evidence . . . 20
3.7.3 The suites and what they establish . . . 20
3.7.4 The board run . . . 20

4 Results and Discussions **27**
4.1 Experimental Setup . . . 27
4.2 Filter Quality and Outlier Removal (Width Sweep) . . . 27
4.3 Processing Cost and Timing Analysis . . . 27
4.4 Parallel Sorting Performance . . . 27
4.5 System Overhead and IPC . . . 27

5 Conclusion and Scope for Further Research **28**
5.1 Conclusion . . . 28
5.2 Scope for Further Research . . . 28

References **29**

List of Figures

2.1 System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem. 5
2.2 Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller. 7
2.3 Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter. 8
2.4 Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region. 10

3.1 How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped. 12
3.2 The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position. 13
3.3 The plan for one 64 KiB block on four workers. Phase 0 sorts four partitions, each into its own slice of the scratch buffer; each later phase merges adjacent runs and writes the other buffer, so the answer alternates between the caller's array and the scratch. A phase cannot begin until every task of the previous one has finished the dashed barriers. 16
3.4 The protocol suite on the board: the wire contract, the shared-region layout and the seal's sensitivity, 186 checks. 22
3.5 The sensor suite on the board. The `sensor:` lines are the test failing on purpose — a missing file, a 3-byte file, a 6-float record — each followed by `ok`. 22
3.6 The block loader suite on the board: the interleaved recording produces the same blocks and the same seals as the host. 23

3.7 The merge sort suite on the board: ordering, the multiset, the plan line and byte-identical output across thread counts. 23
3.8 The median suite on the board, the largest at 7 615 checks: the filter against a reference, across seams, bursts and dropouts. 24
3.9 The shared-memory suite: the results region’s bounds and arithmetic, 13 checks. 24
3.10 The KPI benchmark on the board: sort throughput at one, two and four workers, and the global median computed two ways agreeing bit for bit. 25
3.11 The client deadline test: with no server the client gives up at ten seconds, and with a server that starts three seconds late it connects by itself. 25
3.12 The soak: twenty consecutive runs, every one completing with identical medians and a stable service time per block. 26

List of Tables

3.1 Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column. 11
3.2 The sensor's interface, including the counters that report the recording and the sensor's own timeline. 14
3.3 The sort alone, per 64 KiB block, against the number of workers. The plan column is what the run reported. 18
3.4 The kinds of evidence the suites use, and why each is convincing. 20
3.5 The six suites. The checks are per suite; the total is the figure quoted for both folds. The KPI suite is a benchmark and is counted separately. 21

Abbreviations

AWGN Additive White Gaussian Noise

BSP Board Support Package

FNV Fowler–Noll–Vo

IOV Input/Output Vector

IPC Inter-Process Communication

MAD Median Absolute Deviation

POSIX Portable Operating System Interface

QNX QNX Neutrino RTOS

RMSE Root Mean Square Error

RTOS Real-Time Operating System

SPM Single Particle Model

List of Symbols

σ Standard deviation (noise parameter)

w Filter width (window size in samples)

f_s Sampling frequency (Hz)

N Number of samples / block size

k Number of partitions / worker threads

Chapter 1

Introduction

1.1 Background and Motivation

Embedded systems used for real-time data acquisition often need to process large amounts of sensor data while meeting strict timing requirements. Multi-core processors help meet these demands by allowing computational tasks to run in parallel. However, making effective use of multiple cores requires proper task distribution and synchronization, especially in systems where predictable execution time and responsiveness are important.

Sensor data may also contain unwanted variations, such as sudden spikes, bursts of abnormal readings, or dropouts caused by electromagnetic interference, sensor faults, or changes in operating conditions. Identifying and removing these outliers is important for obtaining reliable results during statistical analysis. Combining this filtering process with data sorting increases the computational workload. In a Real-Time Operating System (RTOS), these operations must be handled efficiently without affecting the system's ability to respond to inter-process communication (IPC) requests and hardware interrupts.

1.2 Project Objectives

This project focuses on developing a real-time client-server system using the QNX RTOS and deploying it on a Raspberry Pi 4. The system is designed to transfer unsorted floating-point time-series data from a client to a server in blocks of at least 64 KB for processing. The server performs two main operations:

1. **Parallel Sorting:** A parallel Merge-Sort algorithm is implemented using multiple POSIX worker threads. Mutexes and condition variables are used to coordinate thread execution and synchronization during the sorting and merging stages.

2. **Rolling Median Filtering:** A robust rolling median filter (Hampel filter) is applied to identify and remove outliers from the time-series data. The filter uses local median and median absolute deviation (MAD) values to detect abnormal readings while preserving

the underlying signal.

After processing, the server notifies the client through a QNX pulse event containing the calculated median and a descriptor specifying the relevant memory offset. The processed arrays remain in shared memory, allowing the client to access the results without requiring another complete data transfer.

1.3 Workload and Dataset

The system is evaluated using a synthetic dataset generated offline with PyBaMM, a Python-based battery modelling package. The dataset contains battery measurements, including current, voltage, and temperature, representing a simulated discharge cycle. To evaluate the ability of the filtering algorithm to detect abnormal readings, controlled noise and outliers are introduced into the data.

The dataset includes the following types of corruption:

- **Gaussian Noise:** Additive white Gaussian noise is introduced into the measurements to simulate variations in sensor readings. Example noise levels include 2 mV for voltage and 8 mA for current.
- **Impulsive Outliers:** Outliers are introduced in the form of sudden spikes, consecutive bursts, and stuck readings. These outliers are introduced at random and occur at a rate of 1.5%. A ground-truth mask is also maintained to identify the injected outliers and evaluate the filtering performance.

This dataset provides a controlled environment for evaluating the sorting and filtering algorithms under different data conditions. It also allows the results to be compared against the known locations of the injected outliers.

1.4 Report Structure

The remainder of this report is organized into four chapters:

- **Chapter 2 (System Overview)** presents the system architecture, including the block diagram, data flow, and sequence of messages exchanged between the client and server.
- **Chapter 3 (Process Implementation)** explains the implementation of the parallel Merge-Sort algorithm, the rolling median filter, and the QNX IPC mechanisms used for communication and data sharing, including IOV transfers, pulses, and shared memory.
- **Chapter 4 (Results and Discussions)** evaluates the system's performance through thread-scaling experiments, filtering accuracy measurements, and execution-time analysis on the Raspberry Pi 4.
- **Chapter 5 (Conclusion)** summarizes the project outcomes and discusses possible improvements and directions for future work.

Chapter 2

System Overview

2.1 Introduction

This chapter provides a high-level overview of the system architecture. It describes the interaction between the client and server, the flow of data, and the communication mechanisms utilized within the QNX RTOS.

2.2 System Architecture and Block Diagram

The system is based on a client-server model running on QNX. The client reads data from a virtual sensor and offloads it in 64 KB blocks to the server. The server utilizes a multi-threaded approach to sort the data and apply a Hampel filter. The results are written to a shared memory region.

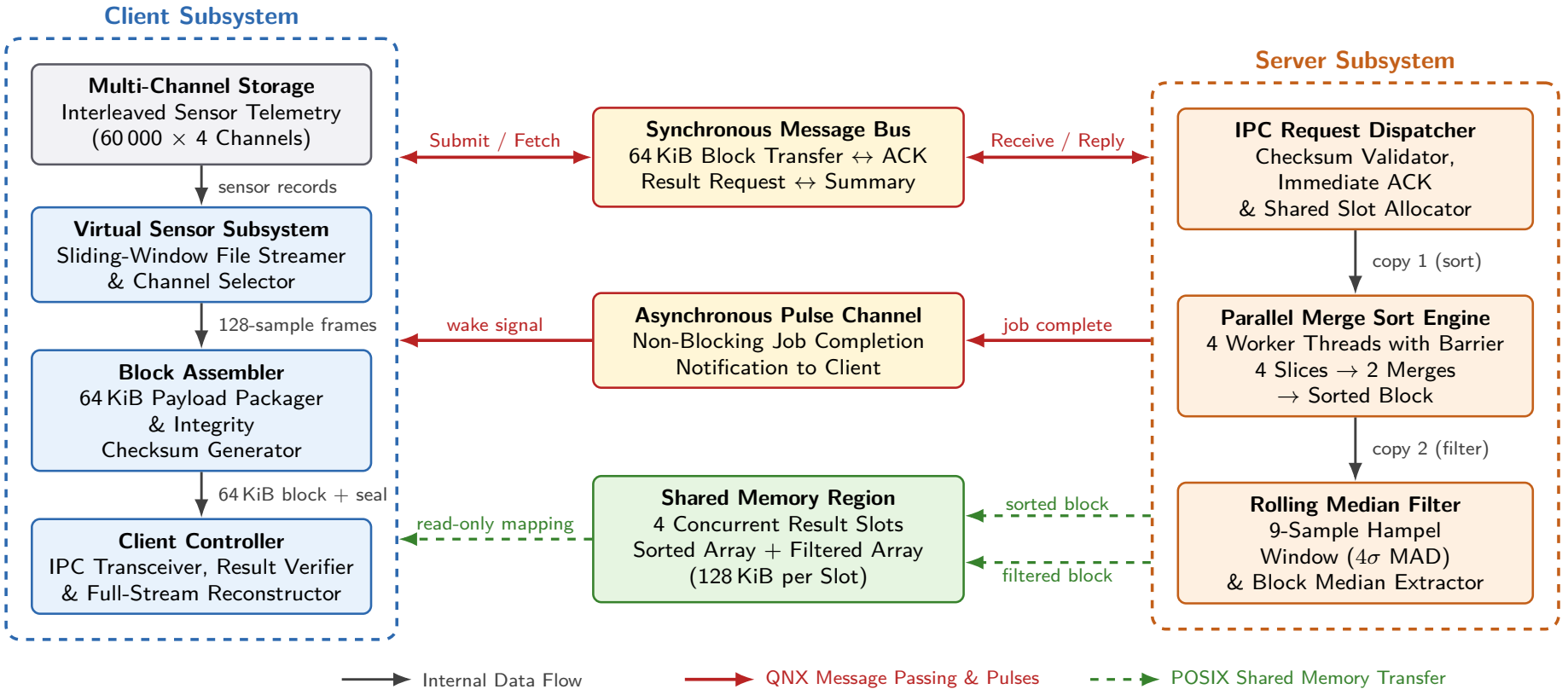


Figure 2.1: System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem.

2.3 Client-Server Communication Sequence

Communication between the two processes follows a strict sequence to ensure data integrity and asynchronous notification.

1. The client connects to the server using `name_open`.
2. The client sends a 64 KB block using an IOV message (`MsgSendv`).
3. The server validates the block, sends an acknowledgement, and queues the job.
4. Once processing is complete, the server sends a QNX pulse back to the client.
5. The client retrieves the result descriptor using a fetch message.

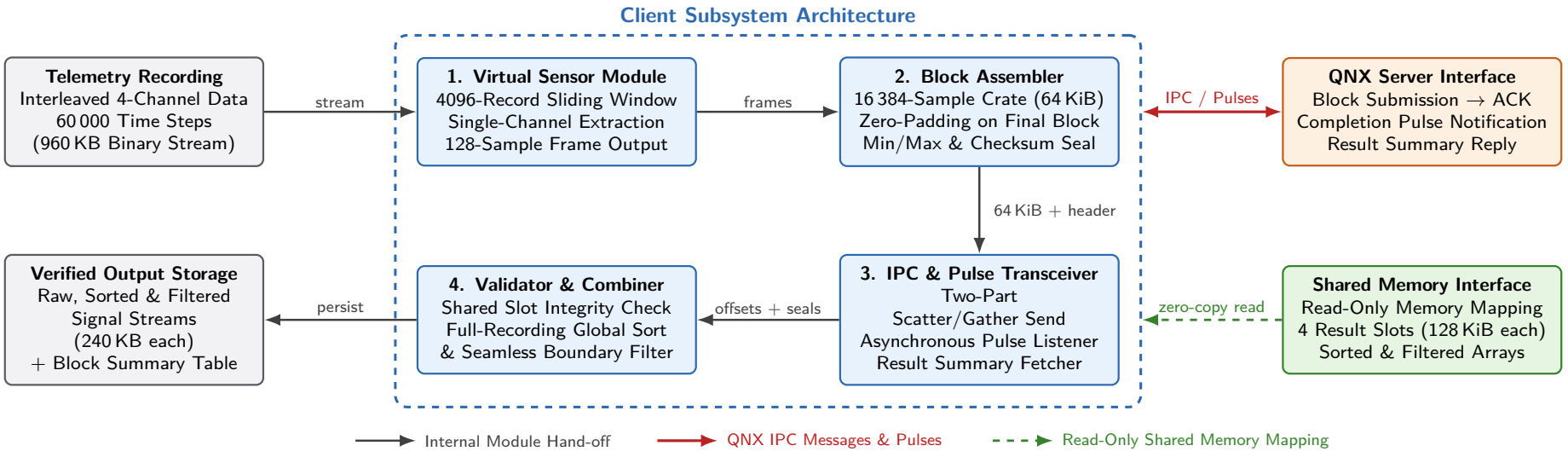


Figure 2.2: Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller.

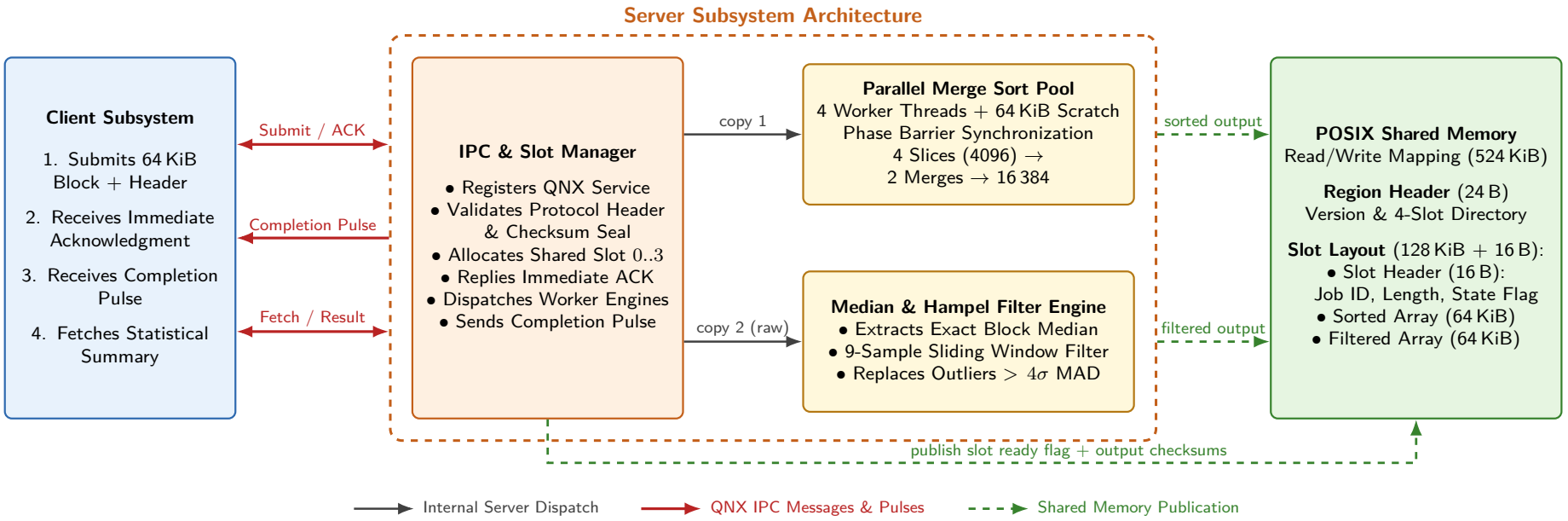


Figure 2.3: Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter.

2.4 Data Flow Diagram

The data starts as raw binary files on disk, is loaded by the client, transferred over the QNX message passing kernel, sorted and filtered, and finally written into a mapped shared memory region.

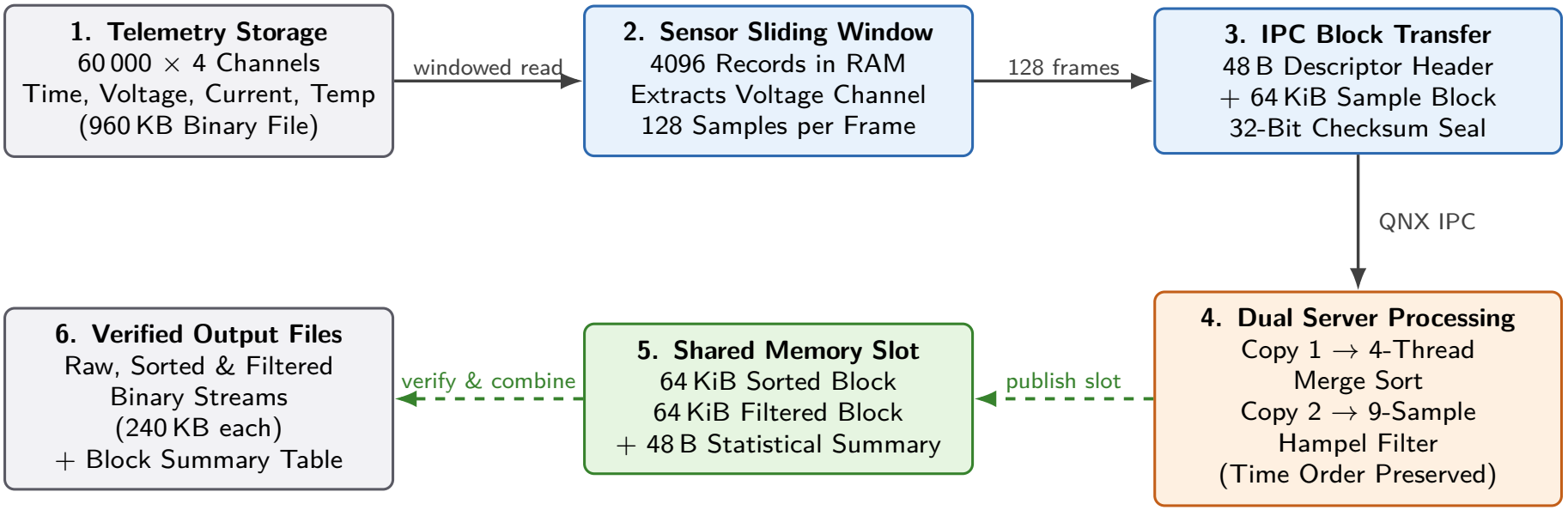


Figure 2.4: Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region.

Chapter 3

Methodology

3.1 Introduction

This chapter details the implementation of the core algorithms, the data structures, and the QNX-specific mechanisms used to build the service.

3.2 Virtual Sensor

There is no physical sensor on the bench, so the service is fed by a module that replays a recording with the behaviour of a live device. Values arrive one sample at a time, in frames, on a clock the module keeps itself rather than at the speed of the host. The purpose is not convenience: with the sensor behind this interface the client and the rest of the pipeline can be developed and tested exactly as they will run against hardware, and a real device can replace the module later without changing anything downstream.

3.2.1 Configuration

Everything the caller can choose sits in one structure, so the two file layouts are configuration rather than separate code paths:

Table 3.1: Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column.

Field	Type	Meaning	Default
`layout`	`int`	`SENSOR_LAYOUT_PLANAR`, one sample per float, or `SENSOR_LAYOUT_INTERLEAVED`, `stride` floats per record	planar
`offset`	`int`	interleaved only: which float of a record this sensor delivers	0
`stride`	`int`	interleaved only: floats per record	1
`frame_samples`	`size_t`	samples a single read may deliver	128
`rate_hz`	`uint32_t`	virtual rate used for timestamps and pacing	100
`loop`	`int`	restart at the end of the recording, as a live sensor would	0

3.2.2 Addressing a sample in either layout

Both layouts are headerless little-endian `float32`, and both are read by one expression, where `next` is the sample the sensor is about to deliver:

$$\texttt{value} = \texttt{window}\big[(\texttt{next} - \texttt{window_first}) \times \texttt{stride} + \texttt{offset}\big] \tag{3.1}$$

A planar file is `stride` 1 and `offset` 0, so sample i is float i. An interleaved file is four floats per record time, voltage, current, temperature so sample i of the voltage channel is float $4i + 1$: record 1 is float 5, byte 20 of the file. The three unused floats of each record are read from disk and discarded; a channel other than the selected one is never returned. Figure 3.1 shows both cases.

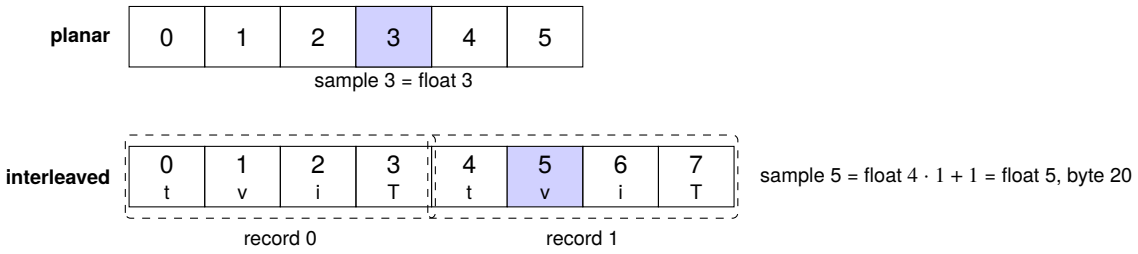


Figure 3.1: How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped.

3.2.3 The read-ahead window

The recording is not loaded whole. Reads pass through a sliding window of `window_samples` records 4096 by default, widened when the caller's frame is larger which is refilled as the reader advances. In the real pipeline the block loader asks for block-sized frames, so the window becomes 16 384 records: 64 KiB of a planar file or 256 KiB of an interleaved one, in one call to `fread`. A caller reading one sample at a time therefore sees exactly the same values as a caller reading whole blocks, a property the tests use as an independent check.

The refill condition is the three-part test in Listing 3.1: the window is empty, the reader was rewound behind it, or it has walked past the end. The loop applies the test before every sample, so no case can read stale data.

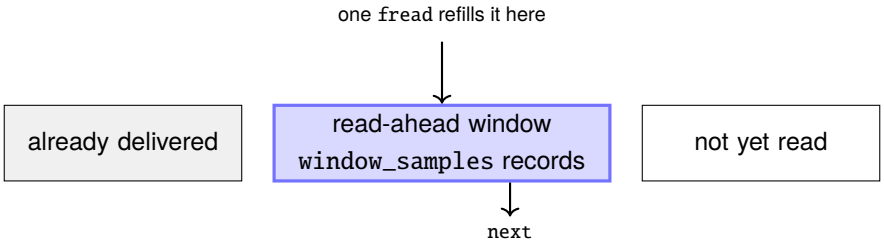


Figure 3.2: The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position.

Listing 3.1: The read loop of `sensor_read()`, trimmed of its error returns. A device read delivers at most one frame; aggregating frames into a block is the caller's job.

```
while (n < cap) {
    if (s->next >= s->total) {
        if (!s->cfg.loop)
            break; /* end of the recording */
        s->next = 0; /* a live sensor never stops */
    }

    if (s->window_count == 0 || /* nothing read yet */
        s->next < s->window_first || /* rewound behind the window */
        s->next >= s->window_first + s->window_count)
        sensor_fill(s, s->next); /* refill at the new position */

    offset = (s->next - s->window_first) * s->cfg.stride + s->cfg.offset;
    dst[n++] = s->window[offset]; /* the one channel that was asked for */
    s->next++;
    s->delivered++;
}
```

3.2.4 Frames, blocks and the sensor's clock

A device read returns a frame, not an arbitrary amount of data: `sensor_read` delivers at most `frame_samples` samples, and `sensor_read_frame` delivers exactly one frame except at the end of a recording. Assembling a 64 KiB payload is the client's concern, which is what `sensor_read_block` does: it calls `sensor_read` until the block is full, so the aggregation sits on the client side of the boundary rather than inside the device.

The clock is arithmetic rather than a system timer. The timestamp of the next sample is

`sensor_index` $\times 10^9 /$ `rate_hz` nanoseconds, computed from a counter of samples delivered, so after 600 samples at 100 Hz it is exactly 6 s however fast the host runs; in loop mode the counter keeps advancing across the wrap, as a sensor that never stops would. `sensor_read_paced` adds a wait after each read so the stream holds the configured rate. That wait and the monotonic clock behind it are the only platform-specific code in the project `clock_gettime` and `nanosleep` on QNX, with one shim for the host build and its resolution is a millisecond, so pacing is meaningful up to roughly 500 Hz.

3.2.5 Rejecting bad input, and the interface

Nothing is assumed about the file. A missing path, an empty file, a length that is not a whole number of `float32` values or of records, an offset outside the record, a stride below one, an unknown layout, a frame or rate out of range each returns a null handle or an error code and prints a sentence naming the problem instead of reading nonsense. Reads follow the same discipline: the number of samples delivered, 0 at the end of a recording, -1 for an error.

Table 3.2: The sensor's interface, including the counters that report the recording and the sensor's own timeline.

Function	Purpose
`sensor_total_samples(s)`	number of samples in the recording
`sensor_remaining(s)`	samples left before the end
`sensor_index(s)`	samples delivered since open or reset
`sensor_open(path, cfg)`	open a recording; a null configuration selects the defaults
`sensor_read(s, dst, max)`	deliver at most one frame
`sensor_read_frame(s, dst)`	deliver exactly one frame
`sensor_read_block(s, dst, n)`	aggregate frames into a block of n samples
`sensor_read_paced(s, dst, max)`	read, then wait for the configured rate
`sensor_timestamp_ns(s)`	the next sample's time on the sensor's clock
`sensor_reset(s)`	rewind and restart the timeline
`sensor_close(s)`	release the handle

3.3 Block Loader

The block loader (`block_loader.c`) aggregates these samples into exact 64 KB blocks (`PROTO_BLOCK_SAMPLES` of 16384 float32 values).

3.4 Parallel Merge-Sort

Sorting a block is the service's one genuinely parallel computation, and it is the step where a mistake is hardest to see: a wrong sort still returns numbers of the right count. The implementation is therefore built around two properties that can be checked rather than argued. The output does not depend on how many workers ran, and the plan the workers actually executed is reported, so a run can be shown to have taken the parallel path rather than the serial one.

3.4.1 The pool and the plan

The threads are created once per process by `ms_pool_create` one per online CPU by default, four on the Raspberry Pi 4, capped at `MS_MAX_THREADS` (64) and they wait on a condition variable when there is nothing to do. Nothing is created or destroyed per block.

Each sort splits the one array it was given according to how much work a worker needs to be worth waking:

$$k = \min\left(\left\lfloor \frac{n}{1024} \right\rfloor, \texttt{nthreads}\right), \qquad \texttt{part} = \left\lceil \frac{n}{k} \right\rceil \tag{3.2}$$

so a 16 384-sample block on four workers becomes four partitions of 4096. Phase 0 posts one partition task per slice; each worker sorts its slice with the ordinary single-threaded merge sort, writing into its own region of one scratch buffer. What comes back is four sorted runs, not one sorted block. The merge phases then halve the number of runs merge adjacent pairs, swap which buffer is read and which is written, repeat until a single run covers the array. Figure 3.3 shows the shape of a block on four workers: three rounds, seven tasks.

3.4.2 The merge, and why the answer is stable

Two sorted runs become one by looking at the front of each and taking the smaller, which is the whole of the merge:

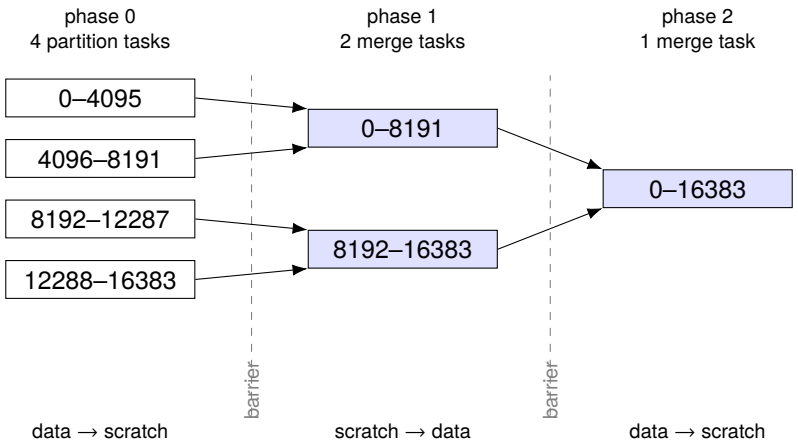


Figure 3.3: The plan for one 64 KiB block on four workers. Phase 0 sorts four partitions, each into its own slice of the scratch buffer; each later phase merges adjacent runs and writes the other buffer, so the answer alternates between the caller's array and the scratch. A phase cannot begin until every task of the previous one has finished the dashed barriers.

Listing 3.2: The merge step. The comparison is <=, so on a tie the value from the left run goes first.

```
static void merge_runs(const float *a, size_t na,
                       const float *b, size_t nb, float *out)
{
    size_t i = 0;
    size_t j = 0;
    size_t k = 0;

    while (i < na && j < nb)
        out[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < na)
        out[k++] = a[i++];
    while (j < nb)
        out[k++] = b[j++];
}
```

That one character, <= rather than <, is what makes the sort *stable*: equal values keep the order in which they arrived. Stability is what makes the result independent of the thread count. A different partition count changes *when* two equal values meet, and the tie rule decides the same way whichever side they come from, so one worker and eight workers produce byte-identical output. The test suite asserts exactly that, for one, two, three, five and eight workers.

3.4.3 The barrier between phases

Each phase's tasks go into one shared array and the workers claim them from a queue. The phase boundary is a barrier on completion, not a delay:

Listing 3.3: `post_phase()`: publish a round's tasks, then block until every one of them has been counted as done.

```
static void post_phase(mergesort_pool_t *p, unsigned ntasks)
{
    pthread_mutex_lock(&p->lock);
    p->ntasks = ntasks;
    p->next_task = 0;
    p->tasks_done = 0;
    pthread_cond_broadcast(&p->work_ready);
    while (p->tasks_done < p->ntasks)
        pthread_cond_wait(&p->phase_done, &p->lock);
    pthread_mutex_unlock(&p->lock);
}
```

The worker side is the other half of the same protocol: take the lock, wait on `work_ready` until a task index is available, copy that index out, and release the lock *before* running the task the sort itself never runs under the lock. When it finishes it takes the lock again, increments the count and the task counter, and the worker that reaches `ntasks` signals `phase_done`. The caller wakes, and only then posts the merges that read what those tasks wrote. There is no sleep anywhere, so a slow machine simply takes longer; it cannot produce a different order. The buffers and the task array are changed only between phases, which is after every task has been counted, so a worker never reads a task that is being rewritten.

3.4.4 Refusals, and the plan line

Two decisions keep the parallel path honest. Inputs below `MS_PARALLEL_MIN` (4096 values) are sorted single-threaded, because waking a pool costs more than the sort saves at that size, and `ms_sort` refuses a second concurrent call on the same pool with -1 rather than corrupting the shared round state one pool sorts one array at a time.

The plan is recorded as it happened, not as it was intended: the statistics count partitions, merge phases and tasks actually executed. That record is what makes the parallelism checkable. "1 partition, 0 merge phases, 1 task" cannot come out of a four-worker run, so the plan line is

proof that the parallel path was taken, whereas the elapsed time alone would not distinguish a fast serial sort from a parallel one.

3.4.5 Measured scaling

On the Raspberry Pi 4, sorting one 64 KiB block (voltage, thread sweep, three repeats per point):

Table 3.3: The sort alone, per 64 KiB block, against the number of workers. The plan column is what the run reported.

Workers	Plan	Time per block	Speed-up
1	1 partition, 0 phases, 1 task	2.40 ms	1.00×
2	2 partitions, 1 phase, 3 tasks	1.32 ms	1.82×
3	3 partitions, 2 phases, 5 tasks	1.09 ms	2.20×
4	4 partitions, 2 phases, 7 tasks	0.80 ms	3.00×

The separate `test_kpi` benchmark measures the same curve with a best-of-20 policy on its own block: 3.010, 1.631 and 1.011 ms at one, two and four workers, a speed-up of 2.98×, so the two measurements agree. Four workers reach three times the throughput of one rather than four: the partitions are created and the merges are contended for, and the filter that follows is serial.

The headroom is worth stating in the same breath. One block is 163.84 s of signal at the 100 Hz design rate, and it sorts in 0.80 ms roughly two hundred thousand times faster than the data arrives. Even at the recording's own, unevenly sampled rate of about 13.36 Hz, a block spans 1226 s of stream.

3.5 Rolling Median Filter (Hampel Filter)

To remove outliers, the server applies a Hampel-style rolling median filter. The filter calculates the median and Median Absolute Deviation (MAD) of a sliding window. If a sample deviates from the median by more than a configurable threshold (4.0 sigma), it is replaced. This preserves legitimate signal edges while removing sudden spikes.

3.6 QNX IPC Mechanisms

3.6.1 Message Passing and IOV

Large data blocks are transferred using `MsgSendv` (Input/Output Vectors) to avoid copying the 64 KB buffer into a single contiguous message.

3.6.2 QNX Pulses and Asynchronous Notification

Upon completion, the server sends a pulse. Because a pulse value is limited to 32 bits, the float representation of the calculated median is bit-cast directly into the pulse payload. The pulse code carries the Job ID.

3.6.3 Shared Memory

To avoid sending the large processed arrays back over IPC, the server maps a named shared memory region (`/dev/shmem/sortmed-results`). The server writes the sorted and filtered arrays directly into this region, and the client maps it read-only to retrieve the results.

3.7 Testing

Every stage of this service has a plausible-looking failure mode. A sort that loses one value, a filter that replaces a legitimate edge, a checksum computed over the wrong bytes, a shared region read at the wrong offset: each still produces numbers of the right shape, and inspection of the output alone would not settle any of them. The tests are therefore built to compare against an answer computed some other way, and to make the system prove that it refuses what it should refuse.

3.7.1 Two folds, one set of sources

The tests run twice, on two very different machines, from exactly the same source files.

The *host* fold is a set of plain C programs built with `gcc`. It is fast, it can be instrumented, and it can do the two things the target cannot do comfortably: compare against the C library's `qsort`, and feed malformed input on purpose.

The *board* fold cross-compiles the same sources with `qcc -Vgcc_ntoaarch64le` and runs them from the boot image on the Raspberry Pi 4, where the compiler, the C library and

the CPU are all different. The board returns the same counts for the same suites, which is the point: with the code held constant, the only thing left that could differ is the kernel, and that is what the message passing, the pulse and the shared results region exercise. The board run is therefore a confirmation of the host run rather than a separate experiment — a disagreement would point at the environment, and agreement means the behaviour is not an accident of one toolchain.

3.7.2 Four kinds of evidence

Table 3.4: The kinds of evidence the suites use, and why each is convincing.

Kind	Example	Why it is convincing
Two independent routes	planar file versus interleaved record; one sample at a time versus whole blocks	different code paths, so a stride, offset or window-refill mistake makes them disagree
An oracle	`qsort` on the same values	not our code and not our bug
Invariants	output ordered; the multi-set unchanged; seals match; byte-identical output for 1, 2, 3, 5 and 8 workers	catches a class of bug rather than one example: a lost value, a duplicated value, a half-merged phase
Deliberate bad input	a 3-byte file, a 6-float record, a NULL path, one nudged sample, a fetch for an unknown job	proves the system refuses instead of quietly doing something wrong

3.7.3 The suites and what they establish

3.7.4 The board run

The output below is taken from the board's console after `test_outputs.txt` was captured. Each suite prints a line per test and a summary; the summaries are quoted here, with the intermediate lines elided.

Table 3.5: The six suites. The checks are per suite; the total is the figure quoted for both folds. The KPI suite is a benchmark and is counted separately.

Suite	Checks	What it establishes
`test_protocol`	186	the wire contract: 64 KiB blocks, safe message numbers, seal sensitivity, shared-region arithmetic
`test_sensor`	642	a file behaves like a device: both layouts agree bit for bit, frames capped, clock exact, junk refused
`test_block_loader`	116	a stream becomes blocks that survive the trip: split right, padded right, sealed over the right bytes
`test_mergesort`	578	sorted, the same multiset, byte-identical for 1, 2, 3, 5 and 8 workers, plan line as expected
`test_median`	7 615	the Hampel filter against a reference, across block seams, bursts, dropouts and bad arguments
`test_shared_memory`	13	the results region's bounds and arithmetic
Total	**9 150**	0 failed, 0 skipped, on host and on board
`test_kpi`	79	benchmark: sort at 1 / 2 / 4 workers, global median computed two ways

Listing 3.4: The six suite summaries, the client deadline test and the soak, as printed on the board.

```
Shared-memory bounds tests: 13 checks, 0 failures

  186 checks, 0 failed, 0 skipped (protocol)
  7615 checks, 0 failed, 0 skipped (median)
  578 checks, 0 failed, 0 skipped (mergesort)
  642 checks, 0 failed, 0 skipped (sensor)
  116 checks, 0 failed, 0 skipped (block loader)
  79 checks, 0 failures (kpi benchmark)

client-wait-test: both cases pass
soak: 20 runs, all completed, all with identical medians
```

The screenshots that follow are the console as it was captured, one per suite. They are the evidence rather than the summary: the test names scroll past with their verdicts, and the suites that reject bad input print the refusals they provoked — every one of those lines is followed by `ok`.

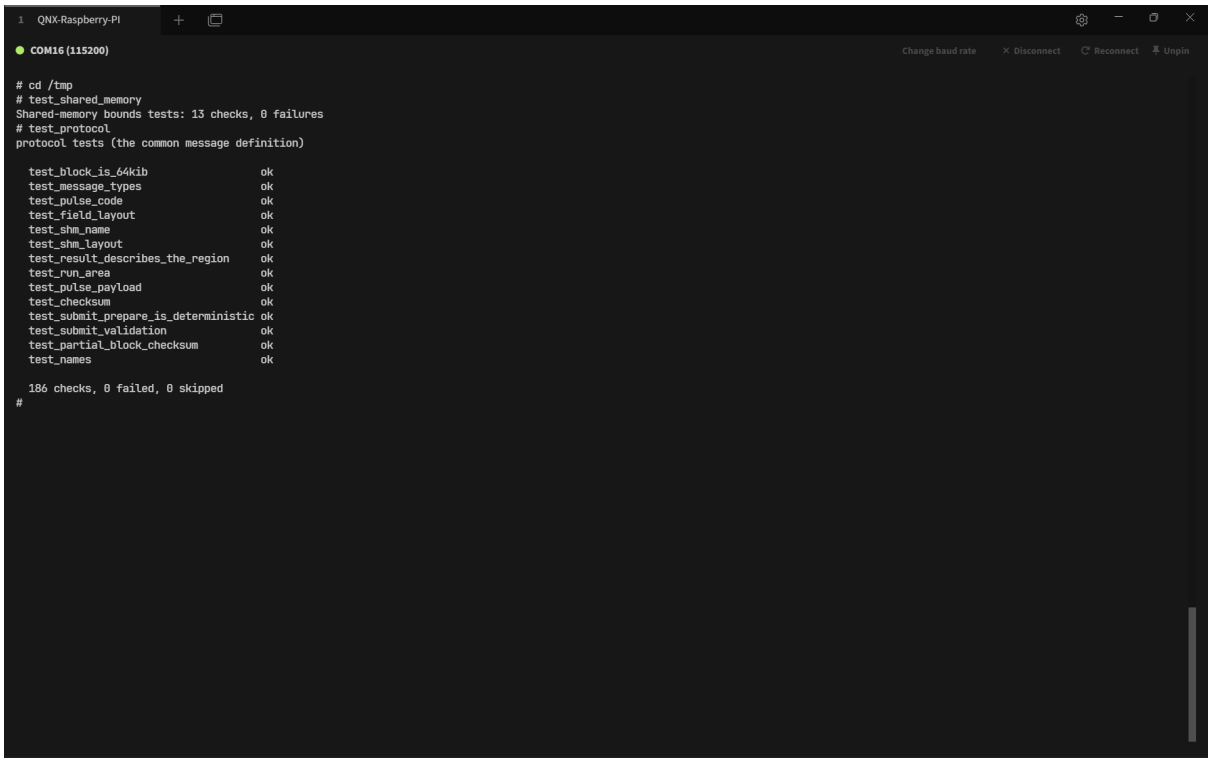


Figure 3.4: The protocol suite on the board: the wire contract, the shared-region layout and the seal's sensitivity, 186 checks.

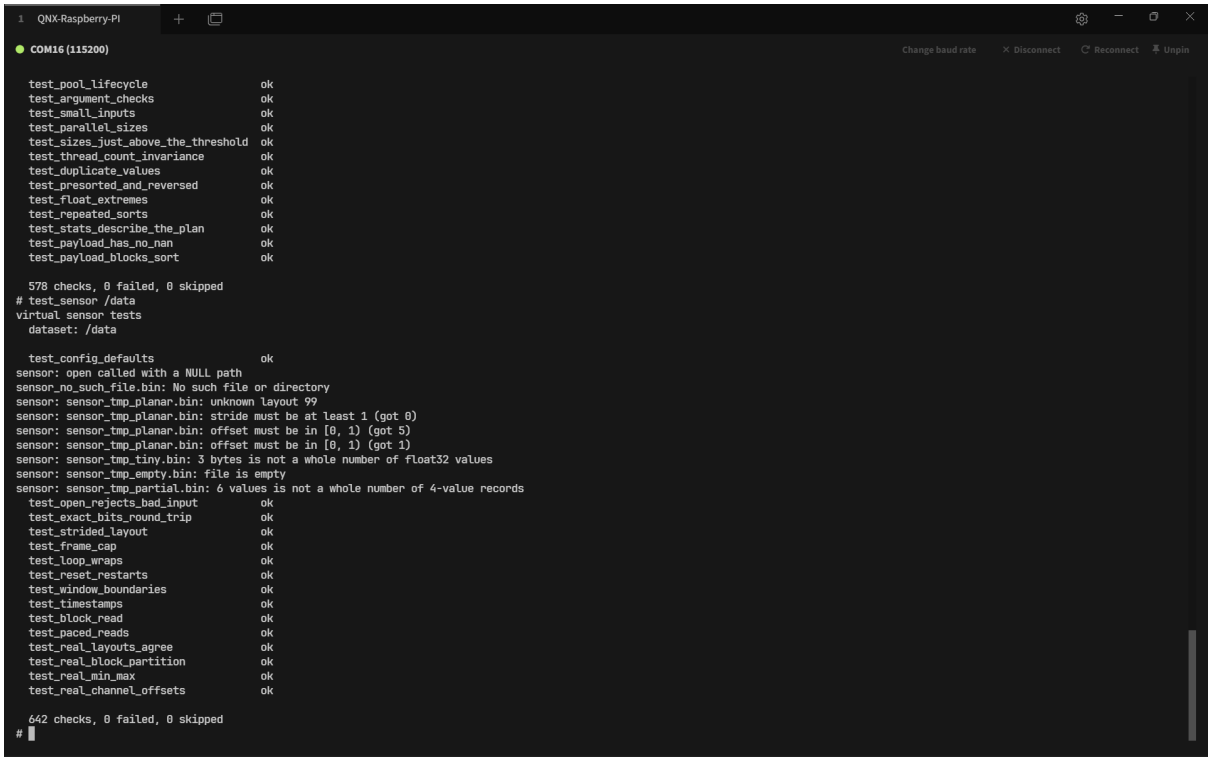


Figure 3.5: The sensor suite on the board. The `sensor:` lines are the test failing on purpose — a missing file, a 3-byte file, a 6-float record — each followed by `ok`.

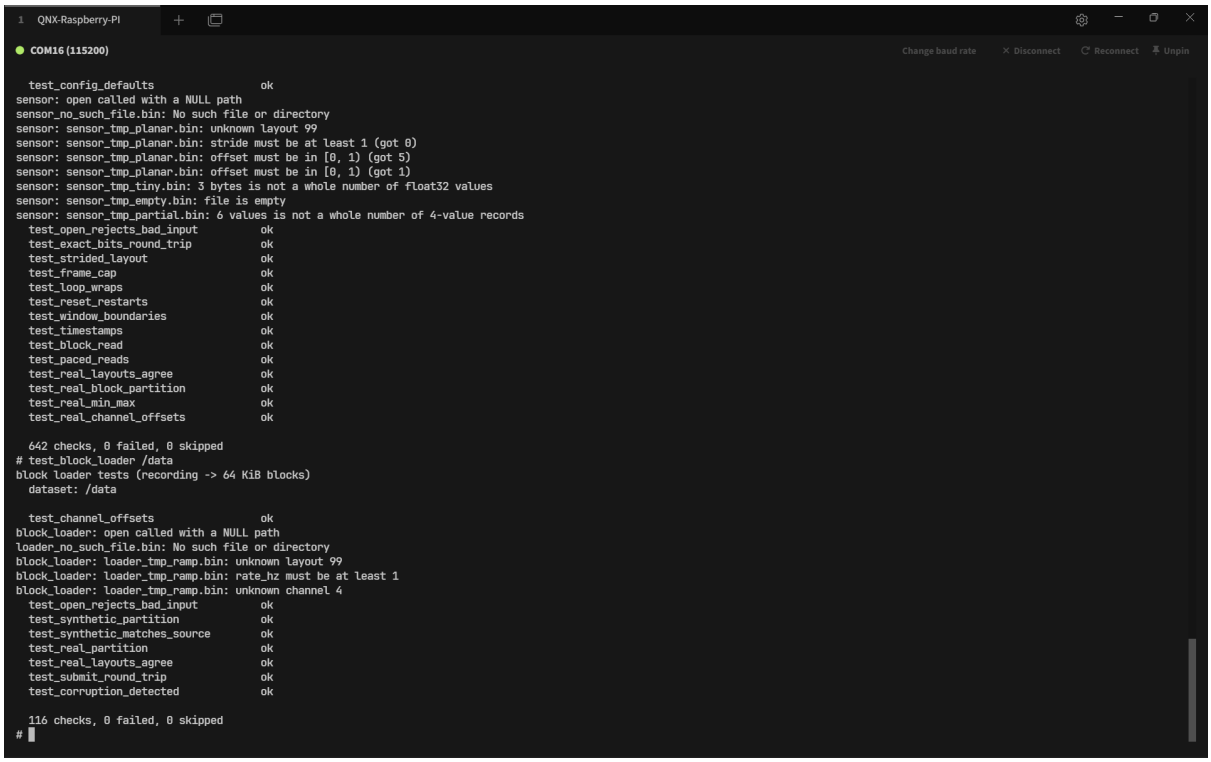


Figure 3.6: The block loader suite on the board: the interleaved recording produces the same blocks and the same seals as the host.



Figure 3.7: The merge sort suite on the board: ordering, the multiset, the plan line and byte-identical output across thread counts.

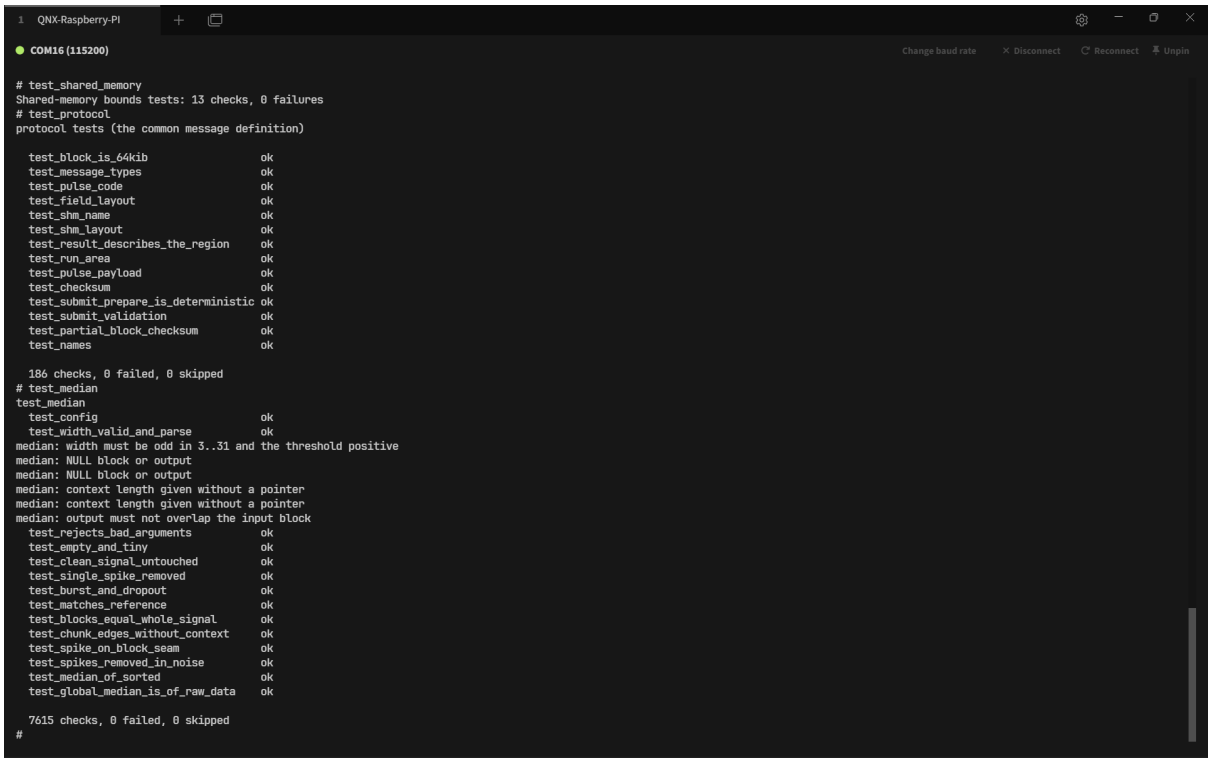


Figure 3.8: The median suite on the board, the largest at 7 615 checks: the filter against a reference, across seams, bursts and dropouts.

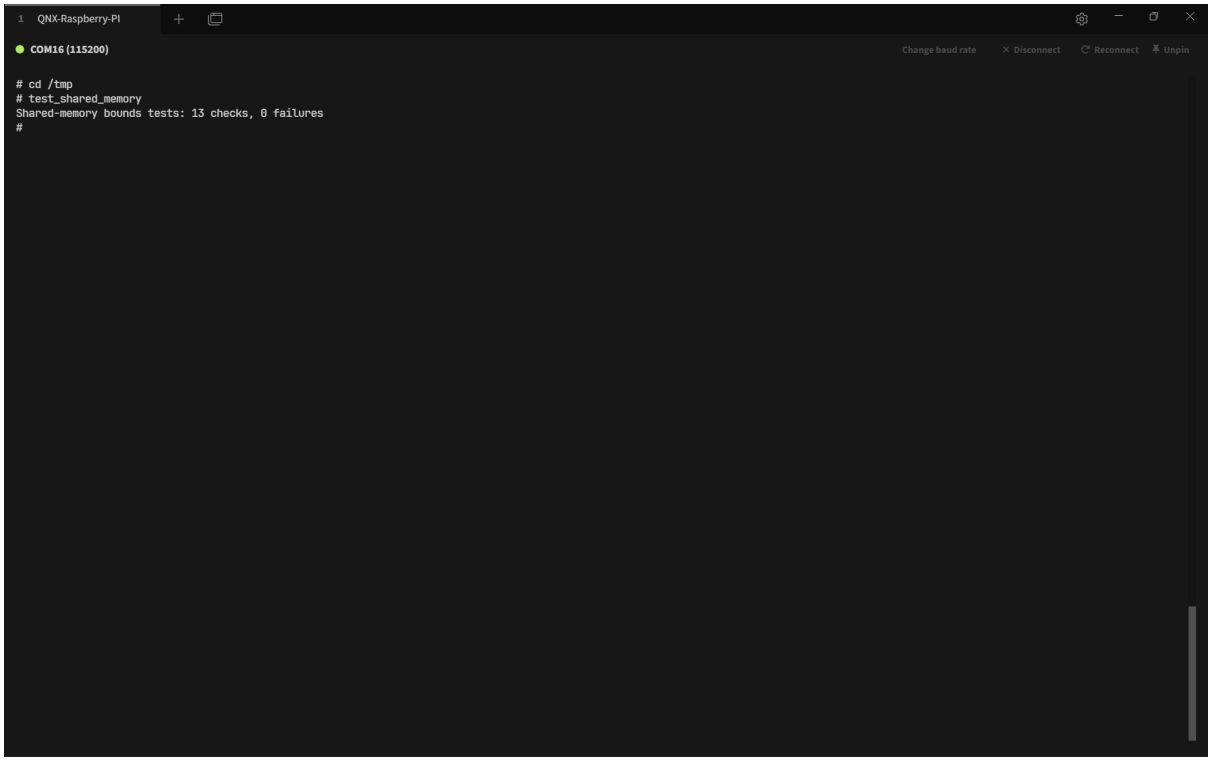


Figure 3.9: The shared-memory suite: the results region's bounds and arithmetic, 13 checks.

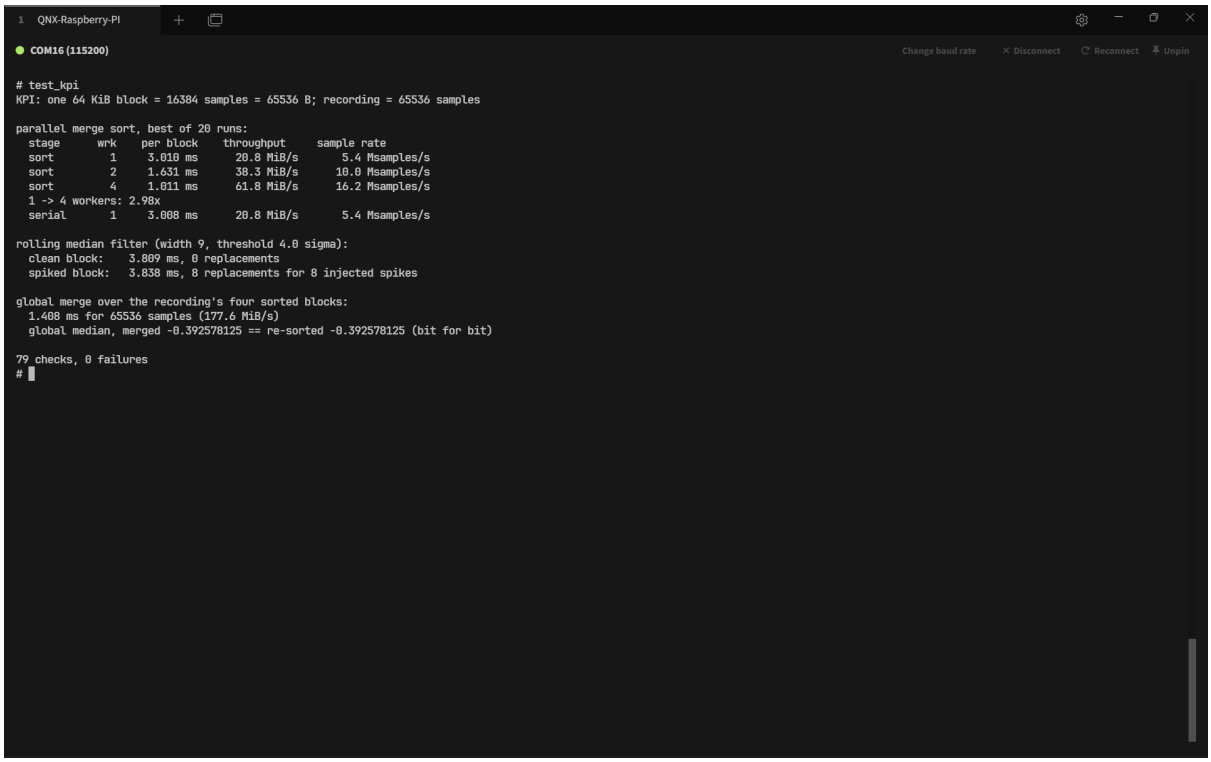


Figure 3.10: The KPI benchmark on the board: sort throughput at one, two and four workers, and the global median computed two ways agreeing bit for bit.

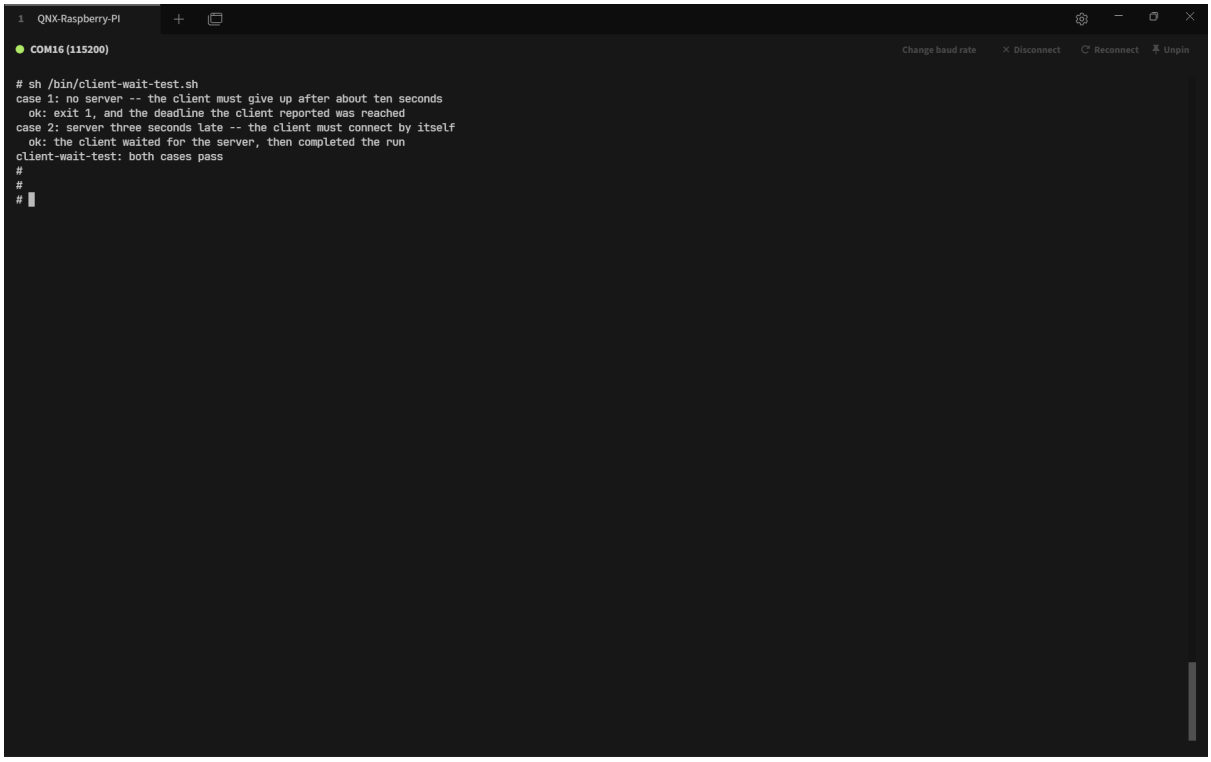


Figure 3.11: The client deadline test: with no server the client gives up at ten seconds, and with a server that starts three seconds late it connects by itself.



Figure 3.12: The soak: twenty consecutive runs, every one completing with identical medians and a stable service time per block.

Chapter 4

Results and Discussions

4.1 Experimental Setup

This section describes the environment and the dataset used for the performance evaluations on the Raspberry Pi 4 board.

4.2 Filter Quality and Outlier Removal (Width Sweep)

This section presents the quality of the Hampel filter across different window widths against the ground-truth mask.

[Insert Figure: width_quality.png Here]

[Insert Figure: error_map.png Here]

4.3 Processing Cost and Timing Analysis

This section discusses the execution time per 64 KB block and how it scales with the filter width.

[Insert Figure: width_cost.png Here]

[Insert Figure: width_tradeoff.png Here]

4.4 Parallel Sorting Performance

This section briefly evaluates the execution time and speedup achieved by the parallel merge-sort on the four cores of the Raspberry Pi 4.

4.5 System Overhead and IPC

This section discusses the latency introduced by the QNX IPC mechanisms (message passing and shared memory).

Chapter 5

Conclusion and Scope for Further Research

5.1 Conclusion

The project successfully implemented a real-time client-server architecture on QNX. The system demonstrated effective parallel sorting using POSIX threads and robust outlier removal using a Hampel filter. The use of QNX pulses and shared memory provided an efficient, low-latency method for data transfer and asynchronous notification.

5.2 Scope for Further Research

Future work could include:

- Integrating a live hardware sensor (e.g., via I2C or SPI) instead of a virtual file-based sensor.
- Implementing dynamic thread allocation based on the current system load.
- Exploring alternative sorting algorithms (e.g., Radix Sort) for floating-point data.
- Adding a graphical user interface (GUI) for live monitoring of the metrics.

References

[1] QNX Software Systems, “QNX Neutrino RTOS Programmer’s Guide,” BlackBerry Limited, 2023.

[2] V. Sulzer, S. G. Marquis, R. Timms, M. Robinson, and D. A. Howey, “Python Battery Mathematical Modelling (PyBaMM),” *Journal of Open Research Software*, vol. 9, no. 1, 2021.

[3] F. R. Hampel, “The influence curve and its role in robust estimation,” *Journal of the American Statistical Association*, vol. 69, no. 346, pp. 383–393, 1974.